

Arquitectura 80x86 / Conceptos necesarios para el manejo correcto de la Arquitectura y su programación

El 8086 es un procesador de 16 bits. Esto significa que trabaja con registros internos de este tamaño y que la amplitud del bus de datos de también de 16 bits. Sin embargo todas las instrucciones que usaremos aquí serán todavía funcionales cuando queramos adaptarnos a generaciones posteriores de procesadores, como el 80386, 486, Pentium, con tamaños de palabra mayores.

Conceptos de espacio de almacenamiento de datos:

Palabra 16 bits, Doble palabra, cuádruple palabra.

Almacenamiento: primero el byte menos significativo y luego el más significativo (en sentido creciente de las direcciones de memoria)

Página 256 bytes

Segmento 64K bytes

Espacio de memoria:

Decimos que un procesador trabaja con un tamaño de palabra N, porque sus registros tienen tamaño N y generalmente las transacciones de datos se hacen en bloques de N bits. Pese a esto el bus que comunica al procesador con el entorno no tiene que corresponder a estas restricciones. Generalmente el bus de datos va a seguir siendo de tamaño N, aunque puede ocurrir que su tamaño sea menor y una transacción haya que realizarla en dos fases (8088). En cuanto al bus de direcciones no está restringido a N líneas, de hecho, interesa que sea lo mayor posible para que abarque el mayor espacio de direcciones posible.

La memoria en el 8086 está dividida en segmentos. Como el tamaño de los registros es de 16 bits, el mayor rango de direcciones que podemos señalar con ellos es de 64K bytes. Sin embargo, el bus de direcciones tiene veinte bits abarcando un espacio de direcciones de 1M bytes. El modo de señalar una dirección entonces consiste en fijar una posición de memoria llamada base e indicar un desplazamiento a partir de esa dirección. Este desplazamiento puede ser como máximo de 64K bytes.

Así toda dirección tendrá dos partes, una que indica la dirección base y otra que indica el desplazamiento (siempre positivo) respecto a esa dirección. Los ejemplos que siguen corresponden todos a la dirección 1050h:

105:0=100:50=10:F50=1:1040=0:1050

A la izquierda del ':' figura el segmento y a la derecha el desplazamiento. El segmento (un valor de 16 bits) no es exactamente la dirección base (que ha de tener 20 bits), para ello hay que desplazarlo todavía 4 bits a la izquierda.

1050 + 0=1000+50=100+F50=10+1040=0+1050

Cuando un programa se va a ejecutar, primero tiene que fijar sus segmentos. Un programa trabaja con cuatro segmentos simultáneamente: el segmento de código, dos segmentos de datos y un segmento de pila. Estos segmentos de memoria, pueden ser disjuntos, o tener parcial o aún completo solapamiento. Las direcciones base de segmentos son siempre múltiplos de 16.

Recursos del procesador:

Registros

Para poder programar en un procesador particular debemos conocer primero el conjunto de recursos que el procesador pone a disposición del programador: registros, interrupciones, espacio de memoria y entrada salida. Y luego el conjunto de instrucciones que nos permiten manipularlos para lograr nuestros fines. Todo esto se analizó en la primera clase pero la perspectiva fue considerando la expansión a la arquitectura 80x386. En esta clase solo trabajamos con 80x86.

En primer lugar se nos ofrecen cuatro registros de 16 bits de propósito general, aunque cada uno tiene asignadas labores más especializadas:

AX: Acumulador

BX: Base

CX: Contador

DX: Dato

Si se trabaja con datos de ocho bits, estos registros pueden ser vistos por las instrucciones como ocho registros de un byte cada uno

AH, AL

BH, BL

CH, CL

DH, DL

SP, BP

SI, DI

Se pueden utilizar como propósito general, aunque tienen una funcionalidad más definida. Son llamados registros de índice y sirven para mediatizar en accesos a memoria. SP y BP se utilizan en accesos al segmento de pila y SI y DI, en los accesos a segmentos de datos.

Para mantener la referencia a los segmentos con los que trabaja un proceso durante su ejecución existen cuatro registros:

CS: mantiene el segmento de código

DS: segmento de datos

ES: segmento de datos extra

SS: segmento de stack

Estos cuatro registros contienen la referencia al segmento correspondiente (desplazándolos cuatro bits a la izquierda la dirección base del segmento) Cualquier referencia a memoria utiliza estos registros de forma automática, aunque el usuario, puede especificar concretamente qué registros desea usar en transacciones determinadas.

CS:IP

DS:BX,SI,DI

SS:SP,BP

ES:BX,SI,DI

Flags, IP

El registro IP contiene la siguiente instrucción a ejecutar y el usuario no tiene ningún acceso a este registro, sin embargo, con instrucciones de bifurcación está modificándolo implícitamente.

En cuanto al registro de flags, contiene una serie de señales que nos indican el estado del procesador en cualquier momento. Estas señales o banderas nos sirven para dirigir el control del proceso a partir de sucesos previos. Las banderas que se nos ofrecen son las siguientes:

Flags de Estado

CF: de acarreo.

OF: de overflow

ZF: de resultado cero

SF: de resultado negativo

PF: paridad del resultado

AF: auxiliar (corrección en lógica decimal BCD)

Flags de Control

DF: de dirección (en instrucciones múltiples)

IF: de interrupciones

TF: paso a paso.

Interrupciones

El sistema de interrupciones para un programador consiste en un conjunto de posiciones de memoria comenzando en la dirección absoluta de memoria 0, que contienen, cada dos palabras, una dirección de salto a una rutina de atención a la interrupción. (la dirección 0 corresponde a la interrupción 0, la dirección 2 a la interrupción 1, etc). Cuando se invoca una interrupción, sea física, por algún dispositivo, o lógica, algún error de ejecución o invocación de programa, el mecanismo de atención toma de la dirección correspondiente que le indica el número de interrupción, la dirección destino del salto y obtiene de allí la siguiente instrucción a ejecutar. Para crear una rutina de atención a determinada interrupción, el programador tiene que localizar el programa en memoria y luego almacenar en la posición de la interrupción correspondiente la dirección de comienzo del mismo. Naturalmente escribir una rutina de interrupción obliga a respetar ciertos protocolos.

Entrada/Salida

Se dispone de instrucciones específicas de entrada salida. El procesador proporciona un espacio de direcciones de entrada salida separado del de memoria. No existe registro de segmento de entrada salida por lo que se supone que todas las direcciones están en el primer y único segmento.

Instrucciones

Hay 92 tipos de instrucciones. Por tipo entendemos instrucción genérica, por ejemplo mover. Una instrucción genérica puede tener muchos casos según donde residan los datos y el modo de direccionamiento que seleccionemos para señalar un dato en memoria.

Todas las instrucciones pueden trabajar con ambos tamaños 8 o 16 bits. Naturalmente ambos operandos deben ser semejantes para poder realizar una operación entre ellos. En los accesos a memoria, dependiendo de los registros utilizados se codifica la instrucción máquina con tamaño byte o tamaño palabra. Si no se utilizan registros, el tamaño debe venir definidos en la propia instrucción mediante una directiva.

Hay 7 modos de direccionamiento

(En todos los casos se usa AX como podría haberse utilizado cualquier otro registro)

- | | | |
|----|---|------------------------------|
| 1. | MOV AX,BX | De registro a registro |
| 2. | MOV AX, 500 | Inmediato. |
| 3. | MOV AX, TABLA | Uso de variable |
| | TABLA es un símbolo que para el compilador tendrá el mismo sentido que una dirección de memoria especificada directamente. | |
| 4. | MOV AX, [BX] | Indirecto mediante registro. |
| | MOV AX, [BP] | |
| | MOV AX, [DI] | |
| | MOV AX, [SI] | |
| | Los registros encerrados entre corchetes contienen un desplazamiento dentro del segmento con el que se trabaje. El uso de determinados registros asume implícitamente un registro de segmento. En este caso BX, DI y SI asume el registro DS y al usar BP se asume el registro de segmento SS | |
| 5. | MOV AX, [BX]+4 | Relativo a base |
| | MOV AX, [BP]+4 | |
| | Al contenido de BX o BP se le suma el valor especificado para obtener el desplazamiento. | |
| 6. | MOV AL,TABLA[DI] | Directo indexado |
| | MOV AL,TABLA[SI] | |
| 7. | MOV AX,TABLA[BX][SI] | Indexado a base. |
| | MOV AX,TABLA[BX][DI] | |
| | MOV AX,TABLA[BP][SI] | |
| | MOV AX,TABLA[BP][DI] | |

El conjunto de instrucciones se puede clasificar en uno de los siguientes grupos:

- Transferencia de datos
- Aritméticas
- Aritmética de bits
- Transferencia de control
- Operaciones de cadenas (de bytes)
- Interrupciones
- Control del microprocesador

Programación en ensamblador:

Un programa en ensamblador es una especificación de instrucciones en el orden en que deseamos que se ejecuten, además de una especificación de los datos de partida. Para facilitar esta especificación existen las instrucciones ensamblador y además otras herramientas que nos proporciona el nivel de abstracción tales como pseudo instrucciones y directivas.

Las directivas son una serie de comandos que nos permiten controlar las acciones del compilador, tanto las relacionadas con la propia codificación de instrucciones como con la generación de información residual del proceso.

Las pseudo instrucciones son instrucciones en ensamblador que podrían no corresponderse directamente con una instrucción en código máquina.

En un programa hay que especificar los segmentos; como mínimo el segmento de código que es el que almacena el programa. Generalmente un segmento de datos y opcionalmente un segmento de pila y un segmento extra.

El código, naturalmente, va encerrado dentro del segmento de código. Generalmente los datos iniciales se encierran el segmento de datos quedando el extra para apoyo de la codificación. Sin embargo, cada situación requiere sus propias decisiones. Un programa puede carecer de segmentos de datos predefinidos, aunque no de segmento de código.

Sumario de Instrucciones 80x86**Transferencia de datos**

IN	- entrada de bytes o palabra
LAHF	- Cargar AH con las banderas
LDS	- Cargar puntero usando DS
LEA	- Cargar dirección efectiva
LES	- Cargar puntero usando ES
MOV	- Mover
OUT	- Salida de byte o palabra
POP	- Recuperar palabra de la pila
POPF	- Recuperar banderas de la pila
PUSH	- Depositar palabra en la pila
PUSHF	- Depositar banderas en la pila
SAHF	- Almacenar Ah en banderas
XCHG	- Intercambiar dos operandos
XLAT	- Traducir

Instrucciones aritméticas

AAA	- Ajuste ASCII en suma
AAD	- Ajuste ASCII en división
AAM	- Ajuste ASCII en multiplicación
AAS	- Ajuste ASCII en resta
ADC	- Sumar con acarreo
ADD	- Sumar (sin acarreo)
CBW	- Convertir byte en palabra
CMP	- Comparar operandos
CWD	- Convertir palabra a doble palabra
DAA	- Ajuste decimal en suma
DAS	- Ajuste decimal en resta
DEC	- Decrementar en uno
DIV	- Dividir sin signo
IDIV	- Dividir con signo
IMUL	- Multiplicar con signo
INC	- Incrementar en uno
MUL	- Multiplicar sin signo
NEG	- Negar/formar complemento a 2

SBB - Restar con acarreo

SUB - Restar (sin acarreo)

Instrucciones de manejo de bits

AND	- Y lógico
NOT	- Negación lógica
OR	- O lógico
RCL	- Rotación a izquierda a través del bit de acarreo
RCR	- Rotación a derecha a través del bit de acarreo
ROL	- Rotación a izquierda
ROR	- Rotación a derecha
SAL	- Desplazamiento aritmético a izquierda
SAR	- Desplazamiento aritmético a derecha (conservando el signo)
SHL	- Desplazamiento lógico a izquierda
SHR	- Desplazamiento lógico a derecha
TEST	- Comparación lógica de dos operandos
XOR	- O lógico exclusivo

Instrucciones de transferencia del control

CALL	- Llamada a procedimiento
JA	- Saltar si superior
JAE	- Saltar si superior o igual
JB	- Saltar si inferior
JBE	- Saltar si inferior o igual
JC	- Saltar si hay acarreo
JCXZ	- Saltar si CX es cero
JE	- Saltar si igual
JG	- Saltar si mayor
JGE	- Saltar si mayor o igual
JL	- Saltar si menor
JLE	- Saltar si menor o igual
JNA	- Saltar si no es mayor
JNAE	- Saltar sino es mayor o igual
JNB	- Saltar si no es menor
JNBE	- Saltar si no es menor o igual
JNC	- Saltar si no hay acarreo
JNE	- Saltar si no es igual

JNG	- Saltar si no es mayor
JNGE	- Saltar si no es mayor o igual
JNL	- Saltar si no es menor
JNLE	- Saltar si no es menor o igual
JNO	- Saltar si no se produce desbordamiento
JNP	- Saltar si no hay paridad
JNS	- Saltar si no es negativo
JNZ	- Saltar si no es cero
JMP	- Salto incondicional
JO	- Saltar si se produce desbordamiento
JP	- Saltar si hay paridad
JPE	- Saltar si hay paridad par
JPO	- Saltar si hay paridad impar
JS	- Saltar si el signo es negativo
JZ	- Saltar si el resultado es cero
LOOP	- Bucle hasta que acabe contador
LOOPE	- Bucle mientras igual
LOOPNE	- Bucle mientras no igual
LOOPNZ	- Bucle mientras resultado no cero
LOOPZ	- Bucle mientras resultado cero
RET	- Retornar de un procedimiento

Instrucciones de manejo de cadenas.

CMPS	- Comparar cadenas de bytes o palabras
CMPSB	- Comparar cadenas de bytes
CMPSW	- Comparar cadenas de palabras
LODS	- Cargar cadena (bytes o palabras)
LODSB	- Cargar cadena de bytes
LODSW	- Cargar cadena de palabras
MOVS	- Mover cadena (bytes o palabras)
MOVSB	- Mover cadenas de bytes
MOVSW	- Mover cadenas de palabras
REP	- Repetir operación de cadena
REPE	- Repetir mientras igual
REPNE	- Repetir mientras no igual
REPZ	- Repetir mientras resultado no cero
REPZ	- Repetir mientras resultado cero

SCAS	- Explorar cadena (b o p)
SCASB	- Explorar cadena de bytes
SCASW	- Explorar cadena de palabras
STOS	- Almacenar cadena (b o w)
STOSB	- Almacenar cadena de bytes
STOSW	- Comprarse una tienda en el polo norte.

Instrucciones de interrupción.

INT	- Invocar a la interrupción
INTO	- Interrupción si desbordamiento
IRET	- Retorno de interrupción

Instrucciones de control de microprocesador.

CLC	- Borrar bandera de acarreo
CLD	- Borrar bandera de dirección
CLI	- Borrar bandera de interrupción
CMC	- Complementar bandera de acarreo
ESC	- Escape (transmitir información a un coprocesador)
HLT	- Parar el procesador
LOCK	- Bloquear el bus
NOP	- No operación
STC	- Activar bandera de acarreo
STD	- Activar bandera de dirección
STI	- Activar bandera de interrupción
WAIT	- Esperar a que termine un coprocesador.

DIRECTIVAS DEL COMPILADOR**Directivas de datos**

Definición de símbolos

EQU - Asigna símbolo a expresión fija

= - Asigna símbolo a expresión variable

Definición de datos

DB - Definir byte

DW - Definir palabra

DD - Definir doble palabra

DQ - Definir cuádruple palabra

DT - Definir diez bytes

Referencias externas

PUBLIC - Definir símbolo público

EXTRN - Definir símbolo externo

INCLUDE - Incluir fichero fuente

Control del ensamblador

END - Fin del módulo fuente

ORIGIN - Origen de código o datos

EVEN - Poner contador de posiciones a par

.RADIX - Definir base de numeración por defecto.

Definición de segmentos y procedimientos

SEGMENT - Comienzo de segmento

ENDS - Final de segmento

ASSUME - Suponer registros de segmentos

PROC - Comienzo de procedimiento

ENDP - Fin de procedimiento

Definición de bloques

GROUP - Agrupar segmentos

NAME - Nombrar un módulo

LABEL - Asignar un atributo a un nombre

RECORD - Definir registro

STRUC - Definir estructura

Directivas condicionales

IF - Evaluar condición y decidir

ELSE - Si condición es falsa

ENDIF - Fin de estructura condicional

Directivas de listado

Formato del listado

PAGUE - Formato de la página del listado

TITLE - Título del listado

SUBTTL - Subtítulo del listado

Listado de macros

.LALL - Listar macros y expansiones

.SALL - Suprimir listado macros y expansiones

.XALL - Listar sólo macros que generan código objeto

Control del listado

.XCREF - Suprimir referencias cruzadas

.CREF - Restaurar listado de referencias cruzadas

.XLIST - Suprimir listado ensamblador

.LIST - Restaurar listado ensamblador

Comentarios

COMMENT - Comentario

Mensajes

%OUT - Emitir un mensaje durante el ensamblaje

Control del listado de los bloques asociados a una condición falsa

.LFCOND - Listar bloques asociados a una condición falsa

.SFCOND - Suprimir ese listado mencionado arriba

.TFCOND - Invertir el modo de listado de los bloques...

Directivas de macro

Definición de macros

MACRO - Comienza macro

ENDM - Fin macro

LOCAL - Define etiquetas dentro de macro

EXITM - Terminar expansión de la macro

PURGE - Borrar macros de la memoria

REPT - Repetir bloque de sentencias un número de veces

IRP - Repetir bloque de sentencias con un valor cada vez

IRPC - Repetir bloque de sentencias con un carácter cada vez

Operadores de macro

& - Operador para concatenar símbolos o texto

;; - Operador para comentarios que no aparecerán en la expansión

! - Operador para interpretar caracteres en sentido literal

% - Operador para convertir una expresión en un número

Programación en ensamblador

El desarrollo de un programa ensamblador se lleva a cabo sobre un simple editor de texto. Ha de trabajarse en texto plano, sin controles, para lo que se recomienda el Block de Notas, de Windows.

A continuación, mostramos un programa ensamblador x86, será el primer ejemplo a utilizar para poder poner en marcha toda la práctica:

```

001.asm
1  datos segment
2  PRIMERO: db 256 dup(?)
3  datos ends
4
5  extra segment
6  extra ends
7
8  pila segment stack
9  dw 128 dup (?)
10 tope label word
11 pila ends
12
13 codigo segment
14 assume ds:datos,es:extra,ss:pila,cs:codigo
15
16 inicio: mov ax,datos
17         mov ds,ax
18         mov ax,extra
19         mov es,ax
20         mov ax,pila
21         mov ss,ax
22         mov sp,offset tope
23
24 ; AQUI VA MI PROGRAMA
25 mov cx,256
26 mov al,0
27 mov bx, offset PRIMERO
28 ciclo: mov [bx],al
29         inc al
30         inc bx
31         loop ciclo
32
33         mov ah,4ch
34         int 21h
35 codigo ends
36 end inicio
37

```

Primero: Definir Segmentos

Segundo: Asociar los registros de segmento y los segmentos

Tercero: Elegido el segmento de código escribir las instrucciones para inicializar los segmentos

Cuarto: Llenar cada segmento con la información correspondiente

Quinto: Indicar el fin y la dirección de comienzo del programa

Si bien los detalles estarán en el video y/o en la clase virtual, se harán algunas aclaraciones. Se define un segmento, indicado por la directiva SEGMENT. Le llamamos DATOS, porque es el que vamos a utilizar como segmento de datos. Dentro de él podemos observar el uso una directiva de reserva de memoria: DB. Esta directiva reserva bytes y existen múltiples maneras de indicar el número. Lo que figura a la izquierda de la directiva es la Etiqueta, que vamos a utilizar dentro del programa para acceder a la primera

posición de memoria que vamos a reservar "PRIMERO". El fin de un segmento viene especificado por la directiva ENDS.

Debajo definimos otro segmento que es el segmento EXTRA de datos. Este segmento tiene funcionalidades particulares dependiendo la instrucción. En particular se lo utiliza para el manejo de cadenas de caracteres.

Debajo definimos otro segmento que va a ser el de pila. No es realmente necesario si no hacemos un uso abusivo de la pila porque el propio ensamblador se encarga de generar un pequeño segmento de pila e inicializar los registros pertinentes. No obstante, lo hemos definido, en cuyo caso nosotros debemos inicializar los registros en el programa.

Aquí usamos la directiva DB, pero para reservar espacio utilizamos 127 DUP('?'), con esto estamos diciendo que reserve 256 bytes y que los inicialice con cualquier caracter. (esto tiene ninguna relevancia en lo que respecta al manejo de la pila, es una cuestión estética)

tope es la etiqueta que vamos a utilizar para inicializar el Puntero de Pila.

Viene a continuación la definición de otro segmento. A este le llamamos código porque va a contener el programa ensamblador.

La directiva ASSUME es un indicador para el ensamblador donde le explicamos para qué vamos a utilizar cada uno de los segmentos definidos. Aquí le estamos indicando que el segmento DATOS va a ser el segmento de datos del programa y que utilice por defecto el registro de segmento DS cada vez que aludamos a una etiqueta de las definidas dentro de este segmento. Lo mismo hacemos para el segmento de código, el segmento extra de datos y el segmento de pila. Ojo: esta directiva no inicializa los registros, salvo el CS, así que habrá que hacerlo en el programa.

El programa

La etiqueta *inicio* marca el comienzo del programa. Es en este punto donde empieza a ejecutarse el programa. Para indicárselo al ensamblador hay que poner al final del archivo una directiva END con el nombre de la etiqueta. Esta acción es estrictamente necesaria para que el procesador empiece a ejecutar el código en este punto precisamente y no desde el comienzo de segmento como lo haría de forma natural.

Las primeras instrucciones que se ejecutan son las que inicializan los registros de segmento. **Observar que lo hacemos mediante el registro AX, esto es porque no es posible mover datos inmediatos directamente a un registro de segmento.**

Si estudiamos los ejemplos propuestos y elevados en la anterior clase podemos ver algunos ejemplos de uso de la interrupción 21h con la función Ah=9h. (LO VEREMOS MAS ADELANTE).

Observar que lo último que se ejecuta es una interrupción 21h con la función AH=4Ch. Esta llamada es la que permite al procesador volver al sistema operativo. Es estrictamente necesario incluir como instrucción final un retorno al sistema operativo, esta es una de las formas, pero hay otras.

Ensamblado

Una vez que tenemos editado el programa debemos ensamblarlo. Para ello utilizamos el comando **tasm**

tasm [/zi] ejemplo.asm

el parámetro */zi* es opcional para el ensamblado, pero necesario si queremos trabajar con el turbo debugger. Este comando devolverá un archivo *ejemplo.obj*, al que someteremos al linkeado mediante el comando **tlink**:

tlink [/v] ejemplo.obj

el parámetro */v* es opcional para el linkado, pero necesario si queremos trabajar con el turbo debugger.

El resultado de este comando es el archivo ejecutable *ejemplo.exe*.

Para ejecutar el archivo invocamos su nombre como un comando.

Turbo debugger

El turbo debugger es una herramienta que nos permite analizar la ejecución del código, su efecto en los registros y la memoria

Para utilizarlo, tenemos que invocarlo con el nombre del ejecutable que queremos analizar como parámetro: *td ejemplo.exe*.